

Tạo cây tiến trình cho chương trình bài tập 3

Đỗ Hoàng Gia Bảo - 23020783

Khoa Điện tử - Viễn Thông

Ngày 22 tháng 3 năm 2026

Tóm tắt nội dung

Báo cáo sẽ trình bày các bước vẽ ra cây tiến trình chứa tiến trình chạy chương trình trong "Bài tập 3" và các tiến trình sinh ra bởi các lệnh `fork()`, hiển thị pid của mỗi tiến trình trong cây, số lượng tiến trình trong cây tiến trình.

Mục lục

1	Yêu cầu bài tập	3
2	Các bước giải quyết bài toán	3
2.1	Phát hiện tiến trình cha và tiến trình con	3
2.2	Thuật toán xây dựng cây tiến trình	4
2.3	Chỉnh sửa chương trình	4

1 Yêu cầu bài tập

Bài tập 3 cho một chương trình như sau:

Mã nguồn 1: Nội dung file C của bài tập

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    int i;
    int N=10;

    for (i = 0; i < N; i++)
        fork();
    return 0;
}
```

Khi chạy chương trình trên, vòng lặp for sẽ chạy 10 lần, mỗi lần đều gọi hàm hệ thống "fork()". Mỗi lần gọi hàm này, số lượng tiến trình tăng lên gấp đôi. Vì ta gọi "fork()" 10 lần trong vòng lặp, chương trình trên sẽ tạo ra 2^{10} tiến trình hay 1024 tiến trình.

Nhiệm vụ của bài tập là phải biểu diễn được tất cả các tiến trình tạo ra từ chương trình trên thành một kiểu cây tiến trình để thể hiện mối quan hệ của các tiến trình. Cần in ra số lượng tiến trình và mỗi tiến trình cần phải in ra số hiệu tiến trình (PID) của chúng.

2 Các bước giải quyết bài toán

2.1 Phát hiện tiến trình cha và tiến trình con

Hàm hệ thống fork() tạo ra một bản sao của tiến trình hiện tại. Giá trị trả về của fork() là chìa khóa để phân biệt:

- Giá trị bằng 0: Tiến trình đang chạy là tiến trình con (child process).
- Giá trị lớn hơn 0: Tiến trình đang chạy là tiến trình cha (parent process), và giá trị trả về chính là PID của con nó.
- Giá trị âm: Lỗi xảy ra, không có tiến trình mới nào được tạo.

2.2 Thuật toán xây dựng cây tiến trình

Để in ra cây tiến trình và số lượng tiến trình một cách chính xác, chúng ta cần sửa đổi mã nguồn gốc. Thay vì chỉ gọi `fork()` và kết thúc, mỗi tiến trình cần:

- Lưu trữ thông tin: Sử dụng một mảng hoặc cấu trúc dữ liệu để ghi lại mối quan hệ cha-con.
- Đồng bộ hóa: Tiến trình cha phải chờ (`wait()`) các tiến trình con kết thúc để đảm bảo cây được in ra đầy đủ và không bị "mồ côi" (orphan process).
- Đệ quy: Cấu trúc cây được hình thành tự nhiên theo mô hình nhị phân. Với $N = 10$, tổng số tiến trình sẽ là $2^{10} = 1024$.

2.3 Chỉnh sửa chương trình

Chương trình được chỉnh sửa như sau để vẽ được cây tiến trình:

Mã nguồn 2: Mã nguồn để vẽ được cây tiến trình

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int i;
    int N = 10;
    pid_t root_pid = getpid();

    // .dot file configuration for tree graph
    if (getpid() == root_pid) {
        printf("digraph_ProcessTree_\n");
        printf("    rankdir=LR;\n");
        printf("    node[shape=circle, fontsize=10];\n");
    }

    for (i = 0; i < N; i++) {
        pid_t child_pid = fork();

        if (child_pid < 0) {
```

```

        perror("fork failed");
        exit(1);
    }

    if (child_pid == 0) {
        // Child process: Print its connection to its parent
        printf("UUUU\"%d\"U->U\"%d\";\n", getppid(), getpid());
    } else {
        // Parent process: continues loop to create more children
    }
}

// Give processes time to finish printing before the file closes
usleep(200000);

// Only the original root process prints the closing brace
if (getpid() == root_pid) {
    printf("}\n");
}

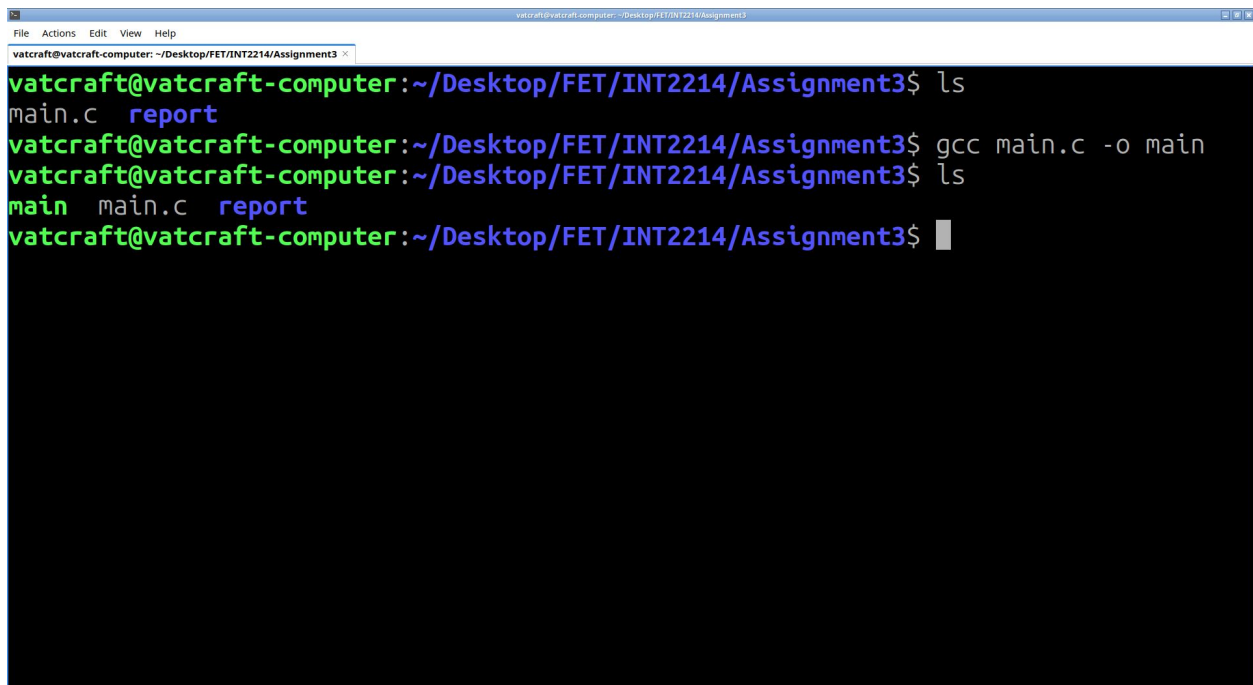
return 0;
}

```

Sử dụng kiểu dữ liệu và định dạng đồ thị thể hiện trong file .dot, các phần mềm như Graphviz có thể được sử dụng để vẽ cây tiến trình.

Sử dụng luồng điều khiển if-else để có thể phân biệt tiến trình con ($pid == 0$) và tiến trình cha ($pid > 0$) để thực hiện in mối quan hệ giữa tiến trình con và tiến trình cha. Sau đó, ghi dòng được in này vào file .dot.

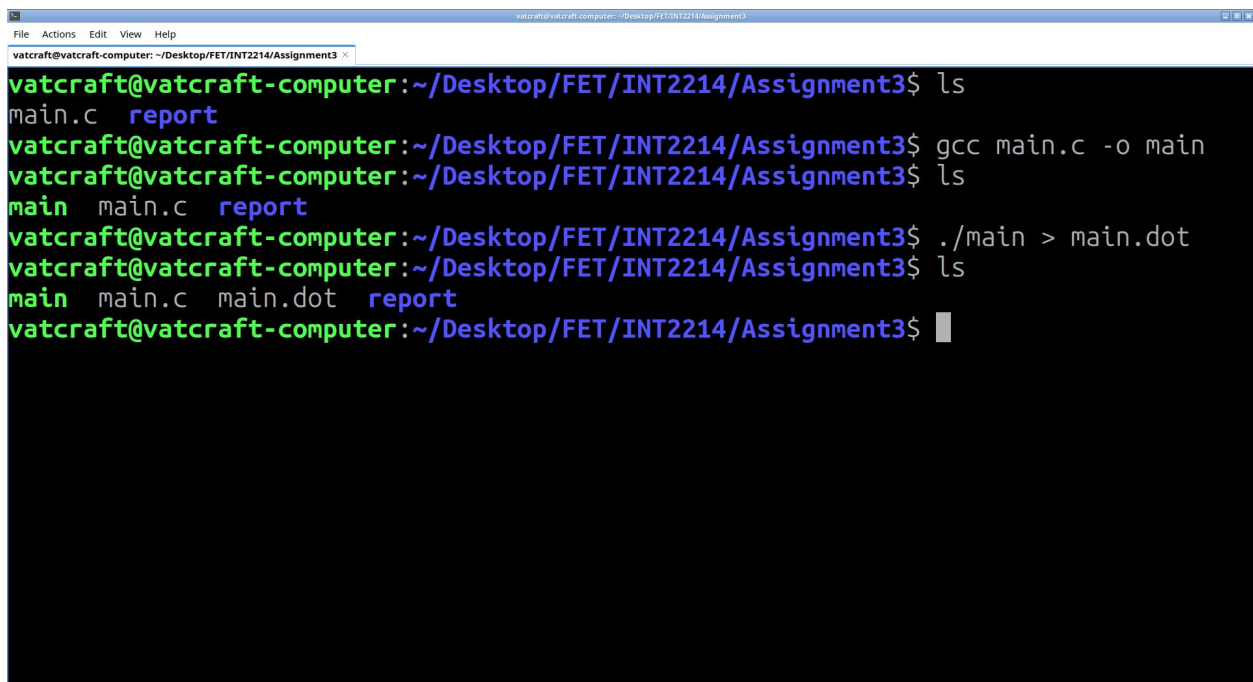
Để chạy chương trình, ta cần phải biên dịch trước bằng cách nhập lệnh sau vào terminal.



```
vatcraft@vatcraft-computer: ~/Desktop/FET/INT2214/Assignment3
File Actions Edit View Help
vatcraft@vatcraft-computer: ~/Desktop/FET/INT2214/Assignment3$ ls
main.c  report
vatcraft@vatcraft-computer: ~/Desktop/FET/INT2214/Assignment3$ gcc main.c -o main
vatcraft@vatcraft-computer: ~/Desktop/FET/INT2214/Assignment3$ ls
main  main.c  report
vatcraft@vatcraft-computer: ~/Desktop/FET/INT2214/Assignment3$
```

Hình 1: Lệnh để biên dịch chương trình

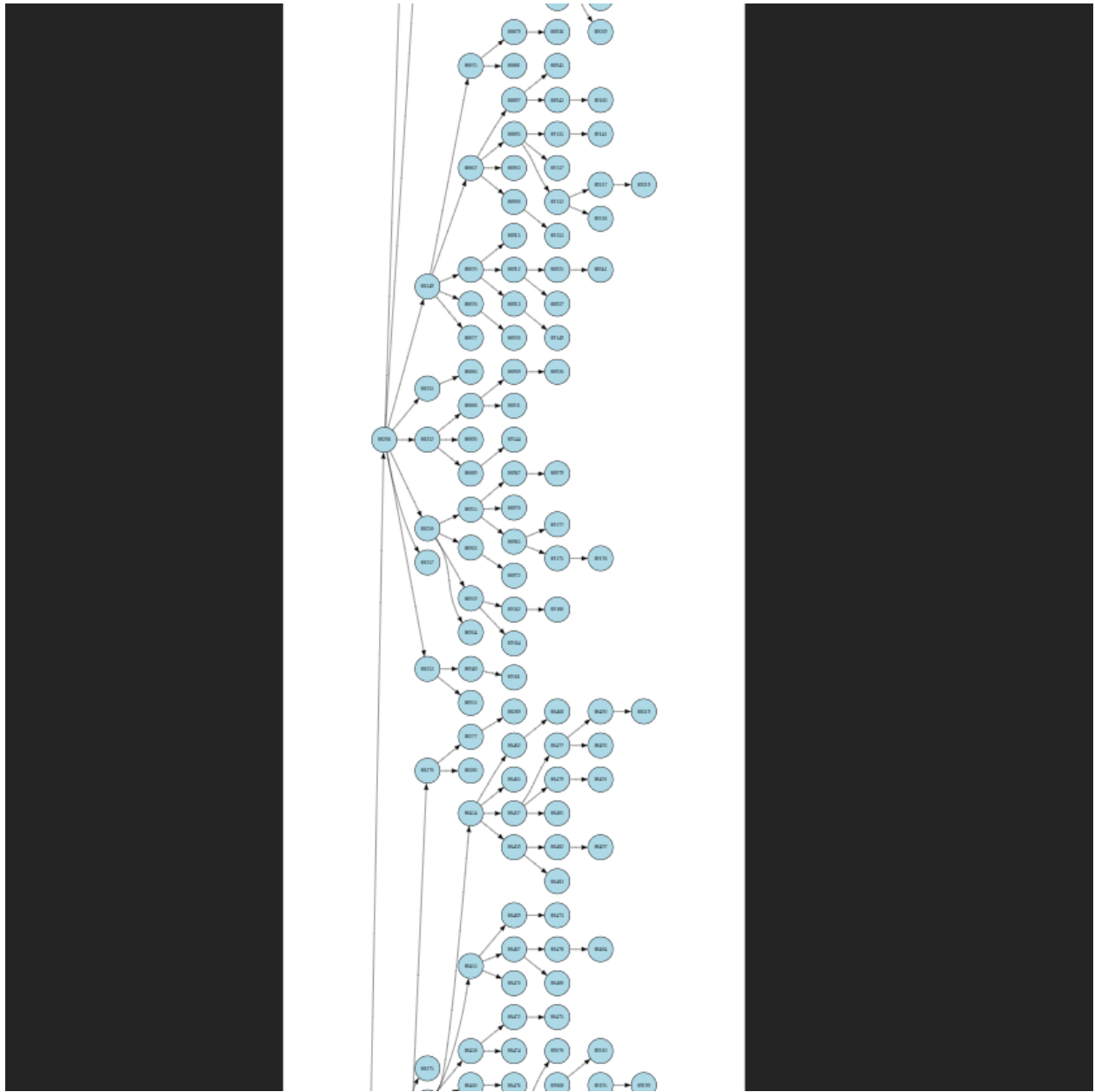
Để có thể viết .dot file, ta nhập lệnh sau vào terminal.



```
vatcraft@vatcraft-computer: ~/Desktop/FET/INT2214/Assignment3
File Actions Edit View Help
vatcraft@vatcraft-computer: ~/Desktop/FET/INT2214/Assignment3$ ls
main.c  report
vatcraft@vatcraft-computer: ~/Desktop/FET/INT2214/Assignment3$ gcc main.c -o main
vatcraft@vatcraft-computer: ~/Desktop/FET/INT2214/Assignment3$ ls
main  main.c  report
vatcraft@vatcraft-computer: ~/Desktop/FET/INT2214/Assignment3$ ./main > main.dot
vatcraft@vatcraft-computer: ~/Desktop/FET/INT2214/Assignment3$ ls
main  main.c  main.dot  report
vatcraft@vatcraft-computer: ~/Desktop/FET/INT2214/Assignment3$
```

Hình 2: Tạo file .dot

Sử dụng phần mềm Graphviz, cây tiến trình được vẽ như sau.



Hình 3: Cây tiến trình sau khi vẽ qua Graphviz

Để có thể nhìn cây tiến trình rõ ràng hơn, mở file graphviz.svg để xem rõ hơn.